

Domain 4

Prompt Engineering & Structured Output

20% of exam · 6 task statements

4.1 Explicit criteria · 4.2 Few-shot prompting · 4.3 Structured output with tool use · 4.4 Validation & retry loops · 4.5 Batch processing · 4.6 Multi-instance & multi-pass review

4.1 Prompts with Explicit Criteria

The foundational principle of Domain 4

The core principle: be explicit

Two words cover this entire domain. Every technique is about removing ambiguity so Claude produces reliable, machine-parseable, consistent output. Vague instructions produce vague output. Explicit instructions produce specific, repeatable results.

Anatomy of a well-structured prompt

```
# Role + task framing
You are a senior code reviewer for a financial services API.
Review the following pull request diff.

# Explicit flag criteria (checkable, not vague)
Flag an issue ONLY when ONE of these conditions is true:
- Claimed behaviour contradicts actual code behaviour
- Missing input validation on user-supplied data
- Unhandled exception that would cause a 500 error
- SQL query vulnerable to injection

# Severity definitions with concrete examples
CRITICAL: data loss or security breach risk
WARNING: incorrect behaviour under edge case
INFO: performance improvement opportunity

# Explicit exclusions — tell Claude what NOT to flag
Do NOT flag: style preferences, naming conventions,
subjective design choices, or performance micro-optimisations.

# Output format constraint
Respond ONLY with JSON. No prose before or after.
```

Vague vs explicit criteria — the exam decision

Vague instructions (unreliable) • 'Be thorough in your review' • 'Be conservative about flagging issues' • 'Make sure it's high quality' • 'Add comprehensive tests' • None of these are checkable — Claude interprets inconsistently	Specific criteria (reliable) • 'Flag only when claimed behaviour contradicts actual code' • 'Every public function must have a unit test' • 'Critical = data loss risk, warning = edge case' • 'Each test must assert exactly one behaviour' • Claude can evaluate each criterion as pass/fail
--	---

XML tags for context organisation

Claude is pre-trained to respond well to XML tags. Use them to separate instructions from user-supplied content — especially important when content could contain instruction-like text:

Flag only critical security issues. Return JSON.

Payment processing service. PCI-DSS compliance required.

{{ code_to_review }}

EXAM TRAP — EXPLICIT CRITERIA

'Be conservative' or 'be thorough' are not evaluable. Claude interprets them inconsistently. Replace with specific categorical definitions: 'Flag only when claimed behaviour contradicts actual code behaviour.'

4.2 Few-Shot Prompting

2–4 concrete examples beat detailed instructions

What is few-shot prompting?

Including 2–4 worked input/output examples in the prompt to show Claude exactly what the desired output looks like. More effective than describing the format in prose because Claude infers the pattern from examples rather than interpreting abstract rules.

KEY CONCEPT

When to use few-shot: whenever the output format is complex or non-standard, when zero-shot produces inconsistent results, or when a description alone doesn't fully convey the desired format. 2–4 examples is the exam-tested sweet spot.

Few-shot example structure

Few-shot examples for code review output

Example 1: Input: "if (user.role = 'admin') {"

Output: {"severity":"critical","issue":"Assignment in condition – use === ","line":14}

Example 2: Input: "const q = `SELECT \* FROM users WHERE id = \${userId}`"

Output: {"severity":"critical","issue":"SQL injection – use parameterised queries","line":28}

Example 3: Input: "// TODO: add error handling"

Output: {"severity":"warning","issue":"Unhandled error path – may throw silently","line":42}

Now review the following code and return a JSON array of findings:

Few-shot design rules

Rule	Why	Guidance
Cover diverse cases	Prevents overfitting to one pattern	Include critical, warning, and info severity examples
Match target distribution	Examples anchor expectations	If 80% of real outputs are warnings, include mostly warning examples

Show exact output format	Eliminates format ambiguity	Include full JSON structure with all required fields
Include edge cases	Teaches boundary handling	Show example where no issue found → return empty array
2–4 examples is optimal	More wastes context with diminishing returns	Start with 3, test reliability, add more only if gaps found

Zero-shot (no examples) • Simple, well-known output formats • Unambiguous tasks • When you want maximum flexibility • Faster to write, less context consumed	Few-shot (2–4 examples) • Custom or non-standard schemas • When description alone doesn't convey format • When zero-shot produces inconsistent results • When consistency across runs is critical
---	--

EXAM TRAP — FEW-SHOT	Using 10+ few-shot examples for a simple extraction task. Overkill — wastes context with diminishing returns. 2–4 diverse examples is the exam-correct answer.
-----------------------------	--

4.3 Structured Output with Tool Use

Guaranteed JSON vs probabilistic prompting

2 approaches — know when to use each

Prompting for JSON directly • Tell Claude to return JSON in the prompt • Simple, no extra API configuration • Risk: prose before/after JSON • Risk: markdown fences (``json) in output • Use for: simple schemas, quick extraction, low stakes	Tool use to enforce schema • Define tool with JSON Schema input_schema • Claude MUST call the tool — structure guaranteed • No prose, no fences, no format deviation • Downstream parsing never breaks on format • Use for: production systems, critical data pipelines
---	--

KEY CONCEPT	Exam rule: when a production system must parse Claude's output programmatically — use tool use with JSON Schema. 'Prompting for JSON' is always a distractor for production reliability scenarios.
--------------------	--

Tool use for structured output — full example

```
// Define a tool with the exact schema you need
{"name": "extract_invoice_data",
 "description": "Extract structured data from an invoice",
 "input_schema": {
  "type": "object",
  "properties": {
    "vendor_name": {"type": "string"},
    "invoice_date": {"type": "string", "format": "date"},
    "total_amount": {"type": "number"},
    "line_items": {"type": "array", "items": {...}}
  },
  "required": ["vendor_name", "invoice_date", "total_amount"]
}

// Force Claude to call it – guarantees structured output
```

```
tool_choice: {"type": "tool", "name": "extract_invoice_data"}
```

JSON Schema field types — know all of them

Type	Use for	Useful constraints
"string"	Text fields, dates, IDs	"format: date", enum: [...], pattern
"number"	Amounts, scores, percentages	minimum, maximum
"boolean"	Flags, present/absent signals	—
"array"	Lists of items	items (schema for each item), minItems
"object"	Nested structures	required: [...] — fields that must be present

Handling missing or uncertain data

When a field cannot be found, Claude should not hallucinate. Design your schema to handle absence:

```
// Make optional fields nullable with a confidence signal

"po_number": {"type": ["string", "null"]},

"confidence": {"type": "string", "enum": ["high", "medium", "low"]}
```

EXAM TRAP —
STRUCTURED
OUTPUT

A downstream system fails because Claude includes markdown fences around the JSON. The fix is NOT a prompt instruction ('don't use markdown'). Fix: tool use with forced tool_choice — structurally guarantees clean JSON every time.

EXAM TRAP —
STRUCTURED
OUTPUT

Using 'prompting for JSON' in a production data pipeline where parsing failures cascade. Production reliability = tool use with JSON Schema, not prompt-based JSON.

4.4 Validation & Retry Loops

Structured validation with targeted feedback

The 4-stage validation-retry loop

- 1

Extract

Claude calls the extraction tool and returns structured JSON output.
- 2

Validate

Your code validates output against JSON Schema + business rules: required fields, types, value ranges, cross-field consistency.
- 3

If invalid → targeted retry

Return the SPECIFIC validation errors to Claude. Never send a generic 'try again' — Claude can't fix what it doesn't know is wrong.
- 4

Cap the retries

Maximum 2–3 retries. If still failing, route to human review — never loop indefinitely.

KEY
CONCEPT

The key insight: retry prompts must include specific validation errors, not generic 'that was wrong, try again.' Targeted feedback gives Claude exactly what it needs to correct the specific field.

Targeted retry prompt — good vs bad

```
// Bad retry (generic – rarely works)

>Your previous response was invalid. Please try again.

// Good retry (targeted – gives Claude what it needs)

>Your previous extraction had the following validation errors:
```

1. invoice_date: missing required field (format: YYYY-MM-DD)

2. total_amount: value -150 is invalid – must be a positive number

Please re-extract and correct these specific fields."

What to validate — beyond JSON Schema

Validation type	What it checks	Example
Schema validation	Types, required fields, format	invoice_date is string in YYYY-MM-DD format
Business rule validation	Domain constraints	total_amount = sum of line_items ± 0.01
Consistency checks	Cross-field relationships	end_date must be after start_date
Completeness checks	Expected fields present	All required fields non-null for this document type
EXAM TRAP — VALIDATION & RETRY	Generic retry: 'Your response was invalid, please try again.' Claude doesn't know what to fix. Always include the specific validation errors.	
EXAM TRAP — INFINITE LOOP	Retry loop with no maximum attempt cap. Always cap at 2–3 retries. Persistent failures route to human review — not infinite loops.	

4.5 Batch Processing Strategies

Message Batches API for large-volume, latency-tolerant workloads

Sequential API calls vs Message Batches API

Sequential API calls • One call at a time, waits for each response • Full price per call • Slow for large volumes • Good for: real-time, latency-sensitive tasks • Good for: small volumes (<50 documents)	Message Batches API • All requests submitted at once, processed async • 50% cost reduction vs standard API • Higher throughput for large volumes • Good for: offline processing, data pipelines • Good for: large volumes (100+ documents)
KEY CONCEPT	When to use Batch API: processing large document sets, overnight data extraction, bulk classification — any workflow where latency doesn't matter but cost and throughput do. NOT for real-time or blocking pre-merge checks.

Batch processing design patterns

Pattern	What it does	When to use
Single-pass batch	Submit all, collect results	Simple extraction with consistent document format
Batch + validation filter	Batch extract, validate each, re-queue failures	When a subset will fail validation
Tiered batch	Fast model for simple docs, capable for complex	When documents vary widely in complexity
Checkpoint batch	Save results incrementally, resume on failure	Very large batches where restart cost is high
EXAM TRAP — BATCH PROCESSING	Using sequential API calls to process 500 invoices overnight. Wrong — slower and costs 2× the Message Batches API. Large-volume + non-real-time = Batches API is always the exam-correct answer.	
EXAM TRAP — BATCH FOR BLOCKING CHECKS	Routing a blocking pre-merge CI check through the batch API. Wrong — developers are waiting for this check. Blocking, synchronous tasks must stay synchronous.	

4.6 Multi-Instance & Multi-Pass Review

Solving attention dilution at scale

Multi-instance architecture (parallel)

- 1

Partition the work
Split input into independent chunks (e.g. files 1–10, 11–20, 21–30). Chunks must be truly independent for this pass.
- 2

Spawn parallel instances
Each chunk processed by a separate Claude call simultaneously. Focused context = deep analysis per chunk.
- 3

Collect all findings
Gather structured output (JSON arrays) from all instances.
- 4

Synthesise
Final Claude call aggregates findings, deduplicates, resolves conflicts, produces unified output.

Multi-pass architecture (sequential) — for dependent passes

Pass	Focus	Input	Output
Pass 1	Per-file analysis	Individual files	Per-file findings JSON
Pass 2	Cross-file integration	All per-file findings	Cross-cutting issues and patterns
Pass 3	Final synthesis	All findings + context	Prioritised, deduplicated report

Extended thinking — when to use it

Use extended thinking for... • Complex architectural decisions • Multi-constraint problem solving • Ambiguous documents requiring deep reasoning • Tasks where accuracy matters more than speed	Don't use for... • Simple, well-structured extraction • Real-time or latency-sensitive tasks • High-volume batch where cost is primary concern • Tasks few-shot examples already handle well
EXAM TRAP — MULTI-PASS	Single Claude call reviewing a 40-file PR with 'be thorough.' Causes attention dilution — shallow coverage of every file. Correct: multi-instance (5–10 files per instance, parallel), then synthesise.

All Domain 4 Exam Traps

Memorise before exam day

EXAM TRAP — EXPLICIT CRITERIA	'Be conservative' or 'be thorough' are not evaluable criteria. Replace with specific categorical definitions: 'Flag only when claimed behaviour contradicts actual code behaviour.'
EXAM TRAP — EXPLICIT CRITERIA	Generic role framing without criteria: 'You are an expert reviewer.' Role alone doesn't fix reliability. Role + explicit criteria + format = reliable output.
EXAM TRAP — FEW-SHOT	Using 10+ few-shot examples for a simple task. 2–4 diverse examples is the exam-correct sweet spot.
EXAM TRAP — FEW-SHOT	Using zero-shot for a non-standard schema when zero-shot is already producing inconsistent results. Add 2–4 targeted examples covering the failing edge cases.
EXAM TRAP — STRUCTURED OUTPUT	Adding 'don't use markdown fences' to fix JSON parsing failures. Not guaranteed. Structurally guaranteed fix: tool use with forced tool_choice.

EXAM TRAP — STRUCTURED OUTPUT	'Prompting for JSON' in a production pipeline where parsing failures cause downstream errors. Production = tool use with JSON Schema.
EXAM TRAP — VALIDATION & RETRY	Generic retry: 'Your response was invalid, please try again.' Include the specific validation errors in the retry prompt.
EXAM TRAP — VALIDATION & RETRY	Retry loop with no maximum attempt cap. Cap at 2–3 retries. Persistent failures route to human review.
EXAM TRAP — BATCH PROCESSING	Sequential API calls for 500 documents overnight. Wrong — slower and costs 2× the Message Batches API.
EXAM TRAP — BATCH FOR BLOCKING	Routing a blocking pre-merge check through the batch API. Wrong — blocking synchronous tasks must stay synchronous.
EXAM TRAP — MULTI-PASS	Single Claude call reviewing a 40-file PR. Attention dilution. Use multi-instance (5–10 files per instance, parallel), then synthesise.
EXAM TRAP — EXTENDED THINKING	Using extended thinking for simple, well-structured extraction. Wastes tokens and adds latency without quality benefit.

Domain 4 complete — 20% of the CCA-F exam. Next: Domain 5 · Context Management & Reliability (15%).